

UNIVERSIDAD DEL NORTE

Facultad de Ingeniería



“AVANCE 2”

Proyecto Integrador: Menina Coiffure (Villa Elisa, Central, Paraguay)

Grupo de Trabajo: Grupo N° 7

Integrantes:

1. Giovanni Rojas
2. Braihan Vega
3. Noelia Serván
4. Danna Sanabria
5. Thiago Rodriguez
6. Lujan Ramirez

Asunción, Paraguay

Carrera: Ingeniería en Informática

Asignatura: Programación IV **Docente:** Prof. Marcos Arrieta Moreira

CONTENIDO

- 1. Introducción y Objetivos
- 2. Metodología Ágil y Reporte de Sprints
 - 2.1. Tablero y sprints
 - 2.2. Flujo Kanban Acumulado (CFD)
 - 2.3. Métricas y cuellos de botella
- 3. Arquitectura de la Solución
- 4. Módulo CRUD Principal: Reservas
 - 4.1. Modelo de datos del módulo
 - 4.2. Reglas de negocio
 - 4.3. API, carga ansiosa y Swagger
- 5. Sesiones Centralizadas
- 6. Docker y Portainer (Código Fuente)
- 7. Evidencia de Pruebas
- 8. Conclusiones y Próximos Pasos
- 9. Referencias

1. Introducción y Objetivos

Menina Coiffure es una peluquería de Villa Elisa que gestionaba sus citas con anotaciones manuales y chats. Esto producía solapamiento de turnos, citas fuera del horario del personal y falta de registro de los precios cobrados.

El proyecto responde con una aplicación web en **Laravel** sobre **PostgreSQL**, orquestada con **Docker Compose** y administrada con **Portainer**, preparada para incorporar **HAProxy**.

El “Avance 2” exige cuatro entregables, cuyo estado al cierre es el siguiente:

Entregable	Estado	Observación
Reporte de Sprints / Flujo Kanban acumulado	Completo	Sprints con flujo acumulado y métricas (sección 2)
CRUD de Reservas con reglas de negocio	Funcional	5 reglas cubiertas por pruebas; HU-03 en validación final
Sesiones centralizadas	Completo	Driver database verificado (sección 5)
Despliegue inicial del Stack en Portainer	Completo	Portainer integrado al stack (sección 6)

2. Metodología Ágil y Reporte de Sprints

2.1. Tablero y sprints

El equipo trabaja con **Scrum** y visualización estilo **Kanban** en Trello. Las columnas son: *Product Backlog*, *Sprint Backlog*, *En progreso*, *En revisión / pruebas* y *Terminado*.

El Diagrama de Flujo Acumulado usa solo las cuatro columnas del sprint, porque el Product Backlog agrupa tareas aún no incorporadas a ningún sprint.

	Sprint 1 (01/09 – 14/09)	Sprint 2 (15/09 – 25/09)
Objetivo	Autenticación, servicios, personal y migraciones base	Reservas, agenda, sesiones centralizadas y stack en Portainer
Tareas	HU-25 login por rol · HU-10 servicios y tarifas · HU-07 personal y especialidades · T-01 migraciones	HU-01 catálogo · HU-02 disponibilidad · HU-03 crear reserva · HU-06 agenda del profesional · T-02 sesiones · T-03 Portainer · HU-07 (<i>arrastrada</i>)
Terminadas	HU-25, HU-10, T-01	HU-01, HU-02, T-02, T-03 y HU-07
Pendientes	HU-07 (en revisión, pasó al Sprint 2)	HU-03 (en revisión) y HU-06 (en progreso)
Cumplimiento	3 de 4 tareas (75 %)	4 de 6 planificadas (66,7 %), más HU-07

2.2. Flujo Kanban Acumulado

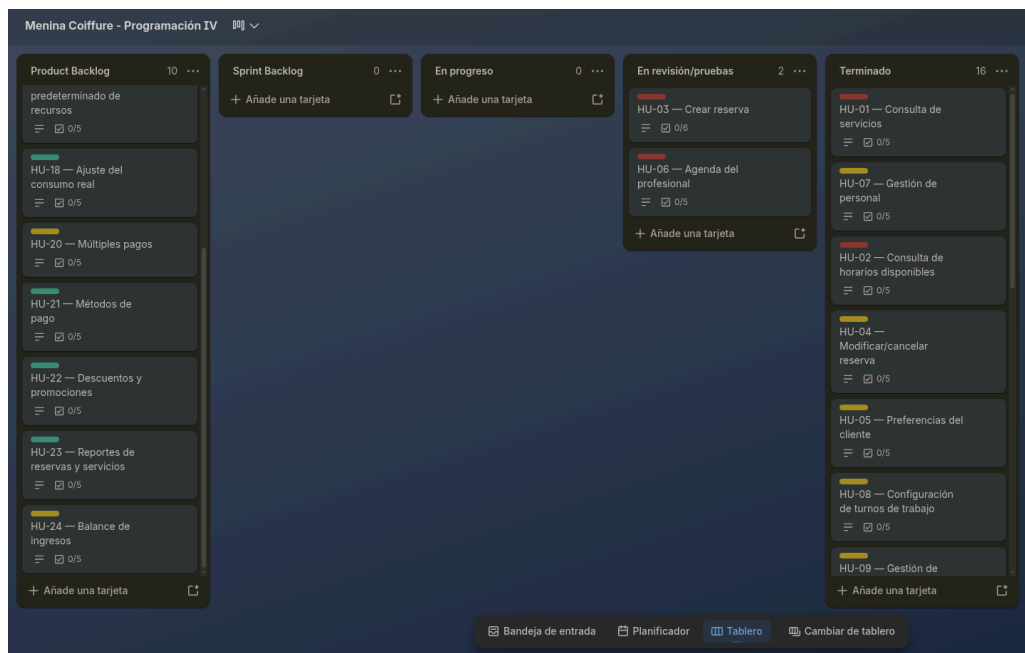
Tarjetas por columna al cierre de cada jornada con movimiento (SB = Sprint Backlog, EP = En progreso, ER = En revisión, T = Terminado).

Sprint 1	SB	EP	ER	T		Sprint 2	SB	EP	ER	T
01/09	2	2	0	0		15/09	4	3	0	0
02/09	2	2	0	0		16/09	4	3	0	0
03/09	2	1	1	0		17/09	4	0	3	0
04/09	2	1	0	1		18/09	4	0	2	1
07/09	1	2	0	1		21/09	1	3	1	2
08/09	1	2	0	1		22/09	1	3	1	2
09/09	1	1	1	1		23/09	0	2	3	2
10/09	1	1	0	2		24/09	0	2	3	2
11/09	0	1	1	2		25/09	0	1	1	5
14/09	0	0	1	3						

Diagrama de Flujo Acumulado

2.3. Métricas y cuellos de botella

Métrica	Sprint 1	Sprint 2
Tarjetas terminadas	3	5
Máximo simultáneo en <i>En revisión</i>	1	3
Cycle Time promedio (8 tarjetas terminadas en ambos sprints)	6,3 días (mínimo 3 días en HU-25 y HU-10; máximo 13 días en T-01)	



En el Sprint 1 la revisión nunca acumuló más de una tarjeta: cada una se resolvía antes de que entrara la siguiente. En el Sprint 2 se formaron dos acumulaciones:

- El **17/09** (HU-07, T-02 y HU-01 entraron juntas a revisión tras iniciar desarrollo el mismo día).
- El **23–24/09** (HU-01, T-03 y HU-02 permanecieron dos jornadas en revisión y se liberaron juntas el 25/09, último día del sprint; HU-01 estaba en revisión desde el 17/09). Además, HU-06 entró a *En progreso* el 23/09 y no llegó a revisión.

Parte de la validación se resolvió sobre el límite del sprint. Como mejora para los Sprints 3 y 4 se propone un **WIP Limit = 2** en *En revisión / pruebas* y no iniciar varias tarjetas el mismo día.

3. Arquitectura de la Solución

Capa	Tecnología
Backend / BD	Laravel (PHP 8.4-FPM) / PostgreSQL 18
Frontend	Vue 3 + Inertia.js + Vite + Tailwind CSS
Servidor web / Contenedores	Nginx (alpine) / Docker Compose + Portainer CE
Pruebas / API	PHPUnit / OpenAPI 3.0 con Swagger UI

Cuatro servicios comparten la red privada `menina_net`:

Navegador `→:8000→` `menina_nginx:9000→` `menina_app:5432→` `menina_db`

(`nginx:alpine`)

(`PHP 8.4-FPM`)

(`PostgreSQL 18`)

Navegador:`9443/:9001→` `menina_portainer-docker.sock→` Motor Docker del host

4. Módulo CRUD Principal: Reservas

4.1. Modelo de datos del módulo

reservas (cabecera: cliente, profesional, fecha, hora de inicio y fin, estado y descuento opcional), detalle_reserva (líneas con servicio_id, precio_aplicado, cantidad y agregado_en_el_momento) y precios_servicio (histórico de tarifas con vigente_desde / vigente_hasta). El precio del catálogo y el precio cobrado no son redundantes: el segundo es un dato propio de la línea de venta.

4.2. Reglas de negocio

La lógica está en `src/app/Services/ReservaService.php` y se ejecuta dentro de `DB::transaction`, de modo que una reserva se guarda completa o no se guarda.

#	Regla	Descripción
1	No solapamiento	Un profesional no puede tener dos citas simultáneas. Hay solapamiento si $\text{inicio_nueva} < \text{fin_existente}$ y $\text{fin_nueva} > \text{inicio_existente}$. Se excluyen las citas canceladas y la propia cita al editar.
2	Turno laboral	La cita debe quedar completa dentro del turno del profesional para ese día de la semana (<code>turnos_personal</code>).
3	Servicios activos	No se puede reservar ni agregar un servicio con

#	Regla	Descripción
		activo = false.
4	Precio congelado	Al crear la cita o agregar un servicio, se guarda el precio vigente en <code>detalle_reserva.precio_aplicado</code> . Cambios posteriores del catálogo no alteran la cita.
5	Inmutabilidad y baja lógica	Una reserva completada no se modifica ni se cancela. Cancelar es pasar el estado a cancelada; el registro se conserva.

Durante la verificación se reforzaron además cuatro casos, todos con HTTP 422 y prueba automatizada: re-cancelar una reserva ya cancelada, modificar una cancelada, agregarle servicios a una cancelada o completada, y cancelar una cita cuya fecha ya pasó.

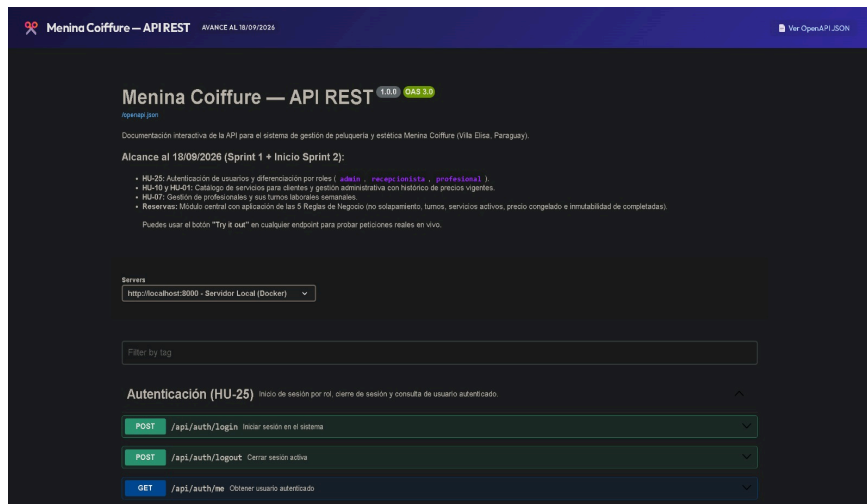
4.3. API, carga ansiosa y Swagger

La API expone listado paginado, consulta, creación, actualización, cancelación y agregado de servicios bajo `/api/reservas`.

Para evitar el problema de consultas N+1, el controlador carga las relaciones de una sola vez y pagina el resultado:

```
Reserva::with(['cliente', 'personal',
'detalles.servicio'])->orderByDesc('fecha')->orderByDesc('hora_inicio')->paginate($perPage);
```

La documentación OpenAPI está en `src/public/openapi.json` y se prueba en `http://localhost:8000/docs`.



5. Sesiones Centralizadas

Con un solo servidor, Laravel guarda las sesiones como archivos locales. Con un balanceador (HAProxy, Avance 3) el usuario puede iniciar sesión en el nodo A y ser atendido luego por el nodo B, que no tiene su archivo de sesión: queda desconectado. Al guardar las sesiones en la base de datos compartida, cualquier réplica consulta la misma fuente y el balanceo es transparente.

`SESSION_DRIVER=database`

`SESSION_LIFETIME=120`

Verificación en PostgreSQL:

```
docker compose exec app php artisan tinker
--execute="print_r(DB::table('sessions')->select('id','user_id','ip_address','last_activity')->limit(3)->get()->toArray());"
```

Existen registros persistidos con identificador, IP y marca de actividad, y la prueba sesiones centralizadas persisten en base de datos valida el comportamiento.

6. Docker y Portainer

Extracto de `docker-compose.yml` (completo en el repositorio): la aplicación espera a que la base esté saludable, y Portainer se integra como cuarto servicio con acceso al motor Docker.

XML

```
services:
  app:
    build: { context: ., dockerfile: Dockerfile }
    container_name: menina_app
    depends_on:
      db: { condition: service_healthy }
  db:
    image: postgres:18
    container_name: menina_db
    volumes: [ "db_data:/var/lib/postgresql" ]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U menina -d menina_coiffure"]
      interval: 5s
      retries: 20
  portainer:
    image: portainer/portainer-ce:latest
    container_name: menina_portainer
```

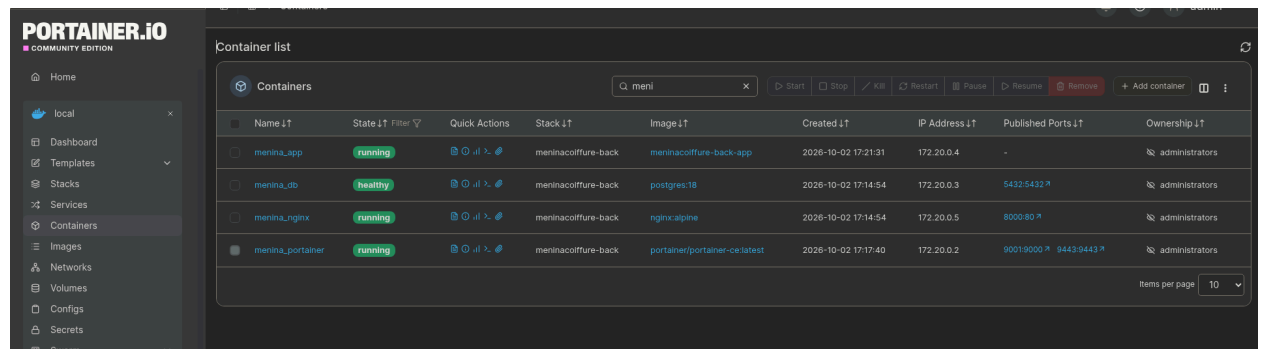
```
ports: [ "9443:9443", "9001:9000" ]
volumes:
  - /var/run/docker.sock:/var/run/docker.sock
  - portainer_data:/data
```

Extracto del Dockerfile:

```
Shell
FROM php:8.4-fpm
RUN apt-get update && apt-get install -y git curl zip unzip
libpq-dev ... \
    && docker-php-ext-install pdo pdo_pgsql pgsql zip mbstring exif
pcntl bcmath gd \
    && apt-get clean && rm -rf /var/lib/apt/lists/*
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
RUN groupadd -g 1000 laravel && useradd -u 1000 -ms /bin/bash -g
laravel laravel
USER laravel
CMD ["php-fpm"]
```

Buenas prácticas: Composer copiado desde su imagen oficial (COPY --from=composer:2), ejecución con el usuario no privilegiado laravel (UID 1000), eliminación de las listas de paquetes en la misma capa para reducir el tamaño de la imagen y Node.js 22 incluido para compilar el frontend. nginx.conf sirve public/, delega PHP a app:9000 por FastCGI y bloquea los archivos .ht*.

Portainer queda disponible en <https://localhost:9443> (HTTPS) y <http://localhost:9001> (HTTP). Las credenciales se crean en el primer ingreso y no se incluyen en este informe. El procedimiento completo de despliegue está en el README.md del repositorio (github.com/Wycie-R/meninaCoiffure-back).



7. Evidencia de Pruebas

La suite de PHPUnit (docker compose exec app php artisan test) ejecutó **19 pruebas con 130 aserciones, todas aprobadas**. Las 11 pruebas del módulo de reservas son:

Prueba	Regla	Resultado esperado
Creación con detalles y precio congelado	3 y 4	HTTP 201
Solapamiento de horarios	1	HTTP 422
Horario fuera del turno laboral	2	HTTP 422
Servicio inactivo	3	HTTP 422
Cancelar reserva completada	5	HTTP 422
Cancelación de reserva no	5	HTTP 200; queda

8. Conclusiones

1. **Cumplimiento.** Se cubrieron los cuatro entregables del Avance 2. HU-03 queda en validación de aceptación final y HU-06 pasa al Sprint 3.
2. **Calidad.** Las 19 pruebas cubren las reglas de negocio y los casos límite de cancelación y edición.
3. **Proceso.** Se propone un WIP Limit de 2 en revisión para evitar acumulaciones al cierre de cada sprint.

9. Referencias

- Laravel. *Documentación oficial*. <https://laravel.com/docs>
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs>
- Docker. *Best practices for writing Dockerfiles*. https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- Portainer. *Portainer Documentation*. <https://docs.portainer.io/>
- OpenAPI Initiative. *OpenAPI Specification v3.0*. <https://spec.openapis.org/oas/v3.0.3>
- Schwaber, K. & Sutherland, J. (2020). *The Scrum Guide*. <https://scrumguides.org>
- Anderson, D. J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.